Structured Onboarding Framework for Security Engineering Teams

Dustin Dishner

EXECUTIVE SUMMARY

Context

Security Engineering teams operate across cloud platforms, identity systems, custom applications, and layered security approaches. In these environments, the consistent application of controls is necessary to ensure the confidentiality, integrity, and availability of systems and data. As these environments grow in scale and complexity, teams bring on new engineers with diverse backgrounds and varying levels of familiarity with architectures, workflows, and threat models. From an operational perspective, effective onboarding is not just a personnel function; instead, it is a foundational security control.

Current Gaps in Onboarding

Despite its importance, onboarding within security engineering teams is often informal, inconsistent, and dependent on undocumented tribal knowledge. New hires often rely on scattered documentation, ad-hoc conversations, and individual initiative to understand critical systems and responsibilities. This creates uncertainty for the new engineer and slows their ability to contribute meaningfully, because they must spend time reconstructing context instead of solving security problems.

Security and Operational Risk

When onboarding lacks structure, the consequences extend beyond slower ramp-up times. Inconsistent knowledge transfer increases the likelihood of misconfigurations in high-impact areas such as identity and access management, cloud security, and detection engineering. In Security Engineering, ambiguity around access, architecture, and responsibility is not neutral; it is risk. It also creates operational fragility by concentrating critical knowledge in a small number of individuals, which increases burnout risk, slows team velocity, reduces morale, and weakens overall security posture.

Framework Overview and Impact

This framework provides a structured, repeatable model that organizations can use to design onboarding plans tailored to their security engineering teams. It establishes a consistent foundation for defining role expectations, documenting workflows, aligning access requirements, and setting milestones that make progress visible to both the new engineer and

the team. By standardizing the onboarding process, organizations can reduce time-to-effectiveness, improve operational resilience, and strengthen the reliability of their security functions. The ultimate objective is not merely documentation, but measurable improvement in role clarity and speed to effective contribution.

Adaptability and Long-Term Sustainability

The framework includes a role-agnostic onboarding structure, templates for defining access and workflows, guidance for role-based onboarding, and a process for building and maintaining onboarding plans over time. Standardization does not eliminate the need for flexibility; rather, it creates a baseline from which thoughtful adaptation can occur. It is intentionally adaptable and should be customized to reflect each organization's architecture, tooling, workflows, and risk profile.

HOW TO USE THIS FRAMEWORK

This framework is meant to give organizations a clear starting point, not a one-size-fits-all onboarding program. The simplest way to use it is to treat each section as a building block and customize only the parts that differ for your specific environment. Start by defining the role you are hiring for and filling out the role definition template, which becomes the anchor for everything else. Next, review the capability boundaries, access requirements, and 30/60/90-day milestones and adjust them to match your systems, workflows, and risk tolerance. From there, map your existing tools, policies, and documentation into the knowledge infrastructure so that you can see what's already in place and what needs to be created. Finally, assign a mentor, plug your customized content into the templates, and sequence the onboarding activities using the cadence outlined in the framework. The goal is not to rewrite the entire document, but to adapt the core structure so your organization ends up with a consistent, repeatable onboarding plan that reflects how your team actually operates.

DEFINE THE TARGET

Effective onboarding starts with a clear understanding of the role a new hire is expected to fill. Security Engineering work varies widely across organizations. This section will help establish the responsibilities, scope, and decision boundaries of the position, which are essential before designing any onboarding activities. By setting the target up front, organizations create the clarity needed to build a focused, role-aligned onboarding plan that supports faster, safer ramp-up.

Onboarding Roles

Manager

The manager sets the direction for the onboarding process and makes sure the new hire understands what the role actually requires. They define expectations, outline the 30/60/90-day milestones, and make sure access, introductions, and early logistics are handled without friction. The manager is also the one who evaluates progress, gives feedback, and makes the calls that carry organizational risk, such as capability boundary transitions, on-call readiness, and alignment with team priorities. Their job is to create clarity, remove blockers, and ensure the new hire is set up to succeed in the long term.

Mentor

The mentor handles the day-to-day side of onboarding and helps the new hire build real capability in the environment. They walk through systems, explain how work actually gets done, review early tasks, and provide the context that does not always show up in documentation. The mentor is the first stop for questions and unblocking, and they help the new hire develop judgement around escalation, risk, and team norms. Their focus is practical, such as making sure the new hire can navigate the work, understand the tools, and build confidence through hands-on support.

New Hire

The new hire plays an active role in their own onboarding. They are expected to engage with the material, ask questions early, and work steadily toward the defined milestones. This includes validating access, completing required training, practicing runbooks, and contributing to documentation as they learn. The new hire should take ownership of their progress by seeking clarity when something does not make sense, applying feedback, and demonstrating sound judgement as their responsibilities expand. Their goal is to build competence, integrate int the team, and become reliable in the areas defined for their role.

Role Definition Framework

For a new hire to understand the role they are meant to fill in the team, the role should have a defined set of responsibilities and competencies that clearly

While there are general requirements for the Security Engineering team as a whole, each new hire was brought on to fill a particular role in the organization, such as cloud security, detection engineering, forensics, incident response. With each role comes a specific set of duties. Clearly establishing the expectations for the role begins with the job posting.

The job posting should contain a listing of the responsibilities, key systems, and required competencies for the position the applicant is expected to deliver on if hired. If there are any items that were more general and not expected or additional items that need to be considered in the exact position hired for, now is a good time to make these updates. It's better to establish the target to shoot for at the beginning rather than trying to hit a moving target.

If you come to this framework after the hiring has been completed, it's not too late to get a good start. Clearly establishing expectations by making sure that a list is provided to the new hire for reference. If the hiring process didn't include a formal list of expectations for the role, just general terms, the template below can be used to create that definition.

Role Definition Template

Roles and Responsibilities

Title	Position Name	Department	Security Engineering
Mission Summary			
A brief description of what the job position entails including contributions the new hire will be expected to make to the team. The language used to fill in this template should always be clear and concise.			
Responsibilities			
• **A bullet list of the responsibilities the new hire will have**			
Key Systems & Competencies			
• **A bullet list of the requirements the new hire must meet to fulfil the needs of this position**			
Required Additional Security			
• **A bullet list of the additional security that people in this role must have to do their job**			

Capability Boundaries

It is very important that the new hire have the ability to demonstrate their capability in a safe manner while building confidence to take on more complex or difficult problems. This framework uses the concept of the traffic light model because it is simple to understand and clearly communicates the risk at each level. I am borrowing the idea from InfraGard who uses the traffic light model to distinguish the allowed level of sharing for their communications. The Traffic Light Model establishes explicit operational boundaries that balance accelerated contribution with controlled risk exposure. The next page presents a table describing the general concept of each color level. Each role should have clearly defined tasks that are slotted into the appropriate color category based on that role's operational risk profile. Movement between categories is milestone-driven, not time-based alone. The model applies to systems, tasks, and decision authority. As with everything else in this framework, this may be modified to conform to expectations within your organization.

The Traffic Light Model for Capability Boundaries

Color	Green	Yellow	Red
Meaning	Safe for early ownership and meant to build confidence and momentum	Require supervision or peer review but build competence in higher-risk activities	Restricted until demonstrated competence or milestone completion to protect critical systems and organizational risk posture
Characteristics	Low production impact Reversible actions Predefined runbooks Limited blast radius	Moderate impact Partial system exposure Change implementation risk Requires contextual judgement	High blast radius Irreversible or difficult rollback Regulatory implications High visibility impact
Examples	Processing standard access requests Updating documentation Triage of low-severity tickets Monitoring dashboard review Executing predefined scripts	Implementing configuration changes with review Reviewing complex access justifications Drafting change requests Handling medium-severity incidents Presenting findings to team	Privileged IAM policy changes Production firewall rule modification Major cloud architecture changes Incident commander role Approval authority for security exceptions
Expectations	Independent execution Clear documentation Daily queue engagement Escalation when unsure	Peer review before execution Shadow-to-lead progression Mentor sign-off required	Explicit authorization Milestone completion (90-day Review) Formal role transition On-call readiness validation
Measurable Indicators & Advancement Criteria	Number of independently resolved tickets Documentation quality review score SLA adherence	Demonstrated consistency in Green Zone Successful completion of 30/60-day milestones Manager approval	

Progressing Levels

Movement from one color to the next is not based solely on time and should reflect capability not tenure alone. It should be based on a demonstrated technical proficiency, documentation quality, escalation judgement, manager/mentor feedback, and performance review milestones.

Purpose & Rationale

Using the traffic light method reduces the ambiguity in what activities the new hire is allowed to do which typically slows contribution. Overexposure creates security risk while underexposure slows development. By creating clear task boundaries, it allows for a safer way to develop capabilities while allowing graduated autonomy to the new hire. This model prevents knowledge hording, makes delegation more predictable, and improves psychological safety because the engineers know their boundaries.

Governance & Oversight

It should be made clear in your organization's customized onboarding framework who is responsible for assigning and reviewing task classification. It should also be agreed on who is the final approval on zone transitions. The documentation should reside in the knowledge management system so that it is able to be reviewed and updated on the same cadence as the other documentation.

30/60/90-Day Performance Milestones

The best way to clearly establish expectations for how the new hire should be contributing is to clearly define performance milestones on some frequency. Most productivity systems build off of the 30/60/90-day planning structure, so I will use that here. Each milestone should have easily achievable goals and clear metrics that can be evaluated to determine if the new hire has met that goal.

Example Milestones:

30-day:

> All required accounts and access provisioned and confirmed
>
> Demonstrates successful navigation and understanding of core systems (ITSM, SIEM, etc.)
>
> Successfully completes 1-2 guided tasks with mentor support
>
> Show comfort navigating team communication channels
>
> Observe one change review meeting and one change readiness meeting

60-day:

Completes intermediate tasks with minimal mentor intervention

Shows the ability to follow runbooks accurately and escalate appropriately

Begins contributing updates to documentation (fixing gaps, clarifying steps)

Demonstrates the ability to support day-to-day tasks in the primary system they will be responsible for

Understands key architecture components and threat model relevant to their role

Shadow 5 tickets under a senior engineer

Present one low-risk change and defend the technical justification. Follow through by appropriately documenting and closing the change ticket

90-day:

Lead 3 tickets independently with review

Take over support for primary system

Participate in 90-day review to align on long-term development goals

Contributes meaningfully to incidents, investigations, or engineering workstreams

Complete one improvement contribution (new documentation, system update, automation, process fix)

BUILD FOUNDATIONAL CLARITY

This section lays the groundwork for everything that follows in the onboarding process. Before a new hire can contribute meaningfully, they need the basics in place, such as having the right access, the right tools, a clear understanding of policies, and a reliable knowledge infrastructure to learn from. The goal here is to remove early friction by making sure the essentials are handled up front and presented in a way that gives the new hire confidence instead of confusion. By establishing fundamentals such as access, training, and documentation early you create the conditions for faster ramp-up, fewer mistakes, and a smoother transition into the team's workflows.

Pre-Arrival Logistics

The manager is responsible for making sure the new hire's first day starts smoothly and without unnecessary blockers. This begins with confirming that all access request, equipment needs, and workspace requirements have been submitted early enough to avoid delays. The manager should coordinate with IT, HR, and any relevant support teams to ensure the laptop, accounts, and communication tools are ready before the new hire logs in. They should also send a brief welcome message that outlines what to expect on the first day, who their mentor will be, and where to go for help. In addition, the manager should prepare the first week's schedule, line up key introductions, and make sure the mentor understands their role and the plan for the initial onboarding cadence. The goal is simple: eliminate avoidable friction so the new hire can focus on learning the environment instead of chasing access or waiting on logistics.

Access & System Provisioning

Access and system provisioning is one of the highest-risk parts of onboarding a Security Engineer, and it is also one of the easiest places for delays and mistakes to pile up. This section lays out a structured approach for getting the new hire the access they need while keeping the process auditable, predictable, and aligned with security best practices. The goal is to eliminate the common pattern of "chasing access for weeks" and replace it with a clear, controlled workflow that gets the engineer productive quickly without exposing the organization to unnecessary risk.

Birthright Access

Birthright access includes the baseline accounts and tools every Security Engineer is given on the first day. These should be requested and provisioned before the new hire arrives to avoid early blockers. At a minimum, this includes access to their network account, communication tools (email and chat), the ITSM system, the documentation repository (read-only at first), and

security policies and standards. These items are foundational and should not require special approvals

Role-specific Access

Role-specific access depends on the engineer's function, such as cloud security, detection engineering, IAM, forensics, etc. This access should be tied directly to the role definition and capability boundaries established earlier in the framework. Examples include access to the SIEM system, EDR console, IAM tooling, cloud portals, and any specialized systems the role depends on. The manager and mentor should confirm that the access list reflects the actual responsibilities of the role, not a generic "security access bundle", which should be included in the birthright access.

Privileged Access and Approval Workflows

Privileged access requires a separate, controlled workflow. Requests should include a clear justification, follow the organization's approval chain, and be logged for audit purposes. Privileged access should never be granted automatically or assumed as part of onboarding. The new hire should not receive elevated permissions until they have demonstrated competence in the relevant systems and completed the required milestones. This aligns with the Traffic Light Model and prevents premature exposure to high-risk actions.

System Access Verification

Once access is provisioned, the new hire must validate that they can log into each system and confirm that the access matches what was requested. The mentor or manager should review the access list with the new hire to ensure nothing is missing and nothing excessive was granted. Any incorrect or unnecessary access should be corrected immediately. Verification should be documented, both for audit defensibility and to prevent future confusion.

Separation of Duties and Least Privileges

Security Engineering roles often intersect with sensitive systems, so it is important to define which roles require strict privilege boundaries and which systems cannot be jointly administered. This section should outline how to prevent conflicting access, how exceptions are documented, and how privilege boundaries are enforced. The goal is to ensure that engineers have the access they need to do their job without creating conflicts of interest or weaking internal controls.

Access Review and Renewal Process

Access should not be "set and forget". This section defines the cadence for reviewing access. These reviews may be done quarterly, semi-annually, or annually depending on the

environment. Reviews should confirm that the engineer still needs each permission, remove anything unnecessary, and document the results. This reinforces least privilege, reduces long-term risk, and ensures the onboarding access decisions stay aligned with the engineer's actual responsibilities.

Core Policy & Compliance Training

Policy and compliance training gives the new hire the foundation they need to operate safely within the organization's governance structure. Security Engineering work carries real risk, and engineers make decisions every day that affect audit posture, regulatory exposure, and the integrity of critical systems. This training ensures the new hire understands the rules that shape those decisions, how those rules show up in daily workflows, and where their authority begins and ends. This section covers how to get the engineer to the point they can explain the policies that govern their work, recognize when something requires escalation, and document their actions in a way that stands up to audit scrutiny. The goal is to build clarity and confidence early so the engineer can contribute without creating unnecessary risk.

Core Security Policies

The new hire should be introduced to the policies that define how the organization manages risk and expects engineers to operate. These policies cover areas such as information security, access control, acceptable use, data handling, secure development, change management, incident response, logging and monitoring, encryption, and cloud security. The point is not to memorize policy language but to understand what each policy governs and how it affects engineering work. During training, the mentor or manager should walk through each policy in plain language, explain where the engineer is responsible for enforcing it, and outline the boundaries of their decision-making authority. This is also the time to highlight common pitfalls, such as places where engineers unintentionally violate policy because they do not yet understand how it applies in practice. A guided walkthrough is far more effective than handing over a stack of documents and expecting the new hire to interpret them alone.

Regulatory and Compliance Landscape

Depending on the organization's industry and data footprint, the Security Engineer may need awareness of regulatory frameworks such as HIPAA, PCI DSS, SOX, GDPR, or FISMA. The purpose of this training is not to turn the engineer into a compliance specialist but to give them enough context to understand why certain controls exist and how their work influences compliance outcomes. Engineers should understand which regulations apply to the systems they support, what evidence auditors expect to see, and how technical decisions can strengthen or weaken compliance posture. This context helps the new hire see the bigger picture behind the controls they implement and reinforces the importance of disciplined documentation.

Risk Management Framework Awareness

Security Engineers should understand how their work maps to established security frameworks such as the NIST Cybersecurity Framework, ISO 27001, MITRE ATT&CK, and the OWASP Top 10. These frameworks shape how organizations structure their controls, measure maturity, and communicate risk. The training should focus on how engineering tasks support these frameworks, where documentation plays a role in demonstrating control effectiveness, and how the engineer's day-to-day work fits into the broader security program. This awareness helps the new hire make better decisions and understand why certain processes exist, even when they may feel procedural or slow.

Decision Authority and Escalation Boundaries

Clear decision boundaries are essential for reducing risk and building confidence. This section defines what the engineer can do independently, what requires manager approval, and what must be escalated to GRC, Legal, or leadership. These boundaries should align with the capability model introduced earlier in the framework. Low-risk, reversible tasks fall within the engineer's independent authority, while medium-risk or ambiguous situations require review. Anything with regulatory, legal, or high-impact implications must be escalated. The goal is to remove ambiguity so the new hire knows exactly when to act, when to pause, and when to ask for help. This clarity prevents accidental policy violations and supports safer decision-making.

Documentation and Audit Discipline

Security Engineering work must be defensible, and that starts with disciplined documentation. The new hire should understand what constitutes sufficient ticket notes, how to record decisions and their rationale, what evidence needs to be retained, and how to prepare for audit requests. This is where you reinforce the principle that "if it is not written, it did not happen." Good documentation is not busy work, but is instead a part of the control surface. It protects the engineer, supports investigations, and ensures the organization can domo9nstrate compliance when required. Setting the expectation early prevents bad habits and establishes a consistent standard across the team.

Delivery Method

Policy and compliance training should be interactive and grounded in real scenarios. A combination of required reading, structured discussion, and case-based walkthroughs works best. The mentor or manager should walk through realistic examples that mirror the engineer's future work, such as handling an access request that violates least privilege or responding to a change that bypassed the established approval process. These scenarios help the new hire practice judgement before they encounter real risk. Recorded modules and policy

acknowledgements can supplement the training, but they should not replace direct conversation. The goal is to ensure understanding, not just confirm completion.

Validation and Competency Checks

Training is complete when the new hire can demonstrate practical understanding, not just recite policy names. They should be able to explain key policies in their own words, navigate a few realistic scenarios, identify escalation triggers, and map a policy to a real workflow. A short comprehension review or scenario-based discussion is usually enough to confirm readiness. This validation step ensures the engineer is prepared to operate within the organization's governance structure and reduces the risk of early mistakes.

Policy-to-Practice Mapping Template

To assist with the policy-to-practice mapping process, I've included a template below that you can add to the knowledge infrastructure for each of the policies that applies to Security Engineering work in your organization.

Policy-to-Practice Mapping Template

Policy Name	The formal name of the policy
Plain-Language Summary	Describe what the policy governs in straightforward, practical terms. Focus on operational impact, not legal language.
Practical Impact on Security Engineer	Explain how this policy affect day-to-day engineering work, including decisions, tasks, and workflows.
Where the Policy Applies	List the systems, processes, environments, or data types where this policy is relevant.
Engineer Responsibilities	Outline what the engineer must do to comply with the policy. Include required actions, prohibited actions, and judgement calls.
Controls That Enforce the Policy	Identify the administrative or technical controls tied to this policy, such as approval workflows, logging, access restrictions, or monitoring rules.
Escalation Triggers	Define the conditions that require escalation to a manager, GRC, Legal, or other team. Keep this section unambiguous.
Comon Failure Points	Note common mistakes or misunderstandings engineers encounter when applying this policy
Related Documentation	Link to runbooks, playbooks, standards, diagrams, or procedures that support this policy in practice.

Knowledge Infrastructure

A key aspect of the onboarding plan that prevents ad hoc sharing of information is to establish an organization and structure for sharing the information important for the team to function. This system will provide access to a searchable collection of support documentation for all of the tools and processes the team supports. It reduces onboarding friction by providing the documented operational context required for a Security Engineer to move from orientation to

contribution without relying on undocumented tribal knowledge. It also reduces single points of knowledge failure, improves audit defensibility, and reduces misconfiguration risk.

In order to define this knowledge infrastructure for your organization, you will need to set the location, structure, and naming conventions that will be used to store the documentation. If the system allows for the use of tags, it is highly recommended to do so based on topic and documentation type to ensure that searching is optimized. If retrieval of documentation is slow due to ambiguity in the structure, it will lead to a less effective system overall.

What to Focus on First

Below is a list of documentation that you should consider adding to the knowledge infrastructure first because everything else builds off of it.

Role documentation

Given the importance of the role definition framework, it should be included first. You should include the role definition template, traffic light model task, 30/60/90-day milestone roadmap, and escalation authority boundaries for the role.

Architecture Documentation

Knowing how your systems are connected and how data flows between them is critical in making security decisions. It's vitally important that up-to-date and thorough network and architecture diagrams are provided for all systems. Most vendors will supply documentation for SaaS systems, but for on-premises systems, you will likely have to develop these on your own. This is also a good place to capture security control integration points, which are commonly used to provide documentation of control implementation for audits. Be sure that every diagram has clear ownership, version control, and review cadence defined. Operating on old information is a sure way to make the wrong decision.

Operational Runbooks & Playbooks

Operational documentation, such as runbooks and playbooks, is the next place I would recommend spending time developing your documentation. Runbooks are step-by-step procedural guides for performing a specific, repeatable task. Playbooks are decision-oriented guides for responding to a scenario where judgement is required. It's important that you cover the most common or critical areas first. A few areas to consider creating runbooks for are access reviews, change implementation, and frequent request processing tasks. Good candidates for playbooks are incident response, security exception processing, and on-call response workflows.

Knowledge Base Articles

The next category of documentation I would recommend spending time on would be knowledge base articles. These cover frequently asked questions, tool quick reference guides, and commonly encountered issues and their resolutions. Also consider including any known false positives and escalation paths.

Policy-to-Practice Mapping

A commonly skipped documentation type is around mapping policies to practical use cases. Creating documentation which lists the policies that the group is responsible for, defining the practical impact they have, where each policy applies, and listing the controls that apply to each is a great starting point satisfying audit requirements. The policy training section includes a template for creating this documentation in a way that is easily digestible and quickly applied.

Ownership & Governance

Governance of the knowledge infrastructure is critical because team operation is only as good as the information they have available. Bad information is worse than no information, because decisions starting from bad information already heading in the wrong direction from the beginning. You should define an owner for each system and set a review cadence on the documentation. This schedule might be quarterly, biannually, or annually depending on the change frequency of the environment. It may also become necessary to trigger additional updates based on major incidents, architectural changes, or tool upgrades.

Quality Control

There should be a review process in the documentation update process to ensure that high quality documentation is being entered into the system. All documentation should have a clear purpose. Step-by-step instructions, decision points, and escalation triggers should be included. Documentation should include screenshots where helpful in understanding the use of the system. Listing the owner and date of last review/update will also be helpful in maintaining the documentation.

Access & Permissions

Access into the knowledge infrastructure will vary based on organization structure. Some teams share documentation among the Security Engineering, Security Operations, Global Risk and Compliance, and Incident Response teams. Others may restrict to only the Security Engineering team. Very few share the sensitive documentation with a wider audience. Some organizations may have a dedicated documentation manager who is the only one able to edit the documentation. It may be worth having varying levels of knowledge sharing where general

troubleshooting documentation is shared more widely and sensitive documents are restricted to only those needing the information. Make sure to have version control and/or an audit trail for changes made to the documentation to prevent accidental loss of documentation. Also, it's not a bad idea to keep an offline copy of the most critical processes that may be needed in the event of a disaster recovery situation.

Onboarding Integration

Day 1 - Include a walkthrough of the documentation repository as part of the first day of onboarding. This underlines the importance of the documentation being available.

Week 1 - In the first week, a guided review of some example documents to show what each type of documentation should include and how to use them.

Week 2 - In the second week, a walkthrough of a required runbook practice exercise to build comfort with the process.

30 days - In the first month, create or update one document to become familiar with the process and create ownership over the system(s) the new hire is going to support.

Measurement & Health Indicators

Some metrics to use to evaluate your knowledge infrastructure are listed below. I recommend choosing one or more to monitor the documentation health and track over time to show improvement.

- Percentage of covered systems with current documentation (updated in last review cycle)
- Average documentation age
- Time-to-answer (how quickly can you find the information you need)
- Percentage of onboarding milestones supported by documentation
- New hire feedback on clarity

DEVELOP OPERATIONAL CAPABILITY

Developing operational capability is the point where onboarding transitions from orientation to execution. At this stage, the goal is no longer exposure to systems or passive learning, but the ability to perform real work safely, consistently, and in alignment with established processes. This section focus on building the skills required to operate within production workflows without creating unnecessary risk, rework, or dependency on senior engineers.

Operational capability is demonstrated through disciplined execution, sound judgement, and the ability to navigate ambiguity using defined escalation paths. New hire should be trained to work the system as it exists, not as they assume it should work. Mastery here is measured by reliability and predictability, not speed.

ITSM Proficiency

Proficiency in the IT Service Management system (ITSM) is foundational for Security Engineering work. Depending on the organization, the ITSM platform may be the system of records for requests, incidents, investigations, changes, and audit evidence. Engineers who cannot operate effectively within it create operational risk, even if their technical skills are strong.

Training should focus on how security work flows through the ITSM, not just how to click through forms. New hires should understand how tickets enter queues, how ownership is assigned, how severity and priority are determined, and how service-level expectations are enforced. This includes recognizing when a ticket should be reassigned, escalated, or converted into a different work item type. The expectation should be clear early that if work is not documented in the ITSM, it is assumed not to have happened.

Engineers should be able to demonstrate:

- Correct interpretation of ticket metadata and impact
- Consistent use of investigative notes and evidence attachments
- Proper escalation and handoff when risk exceeds their authority
- Closure of tickets in a way that is defensible to auditors and future reviewers

Incident and Request Handling

Handling incidents and requests is where judgement is developed and tested. Training should emphasize repeatability, escalation discipline, and alignment with established workflows rather than improvisation.

New hires should be walked through recent real examples to understand how to distinguish false positives from actionable issues, how to evaluate access justifications, and how to

document decisions clearly. This includes reviewing why certain requests were approved or denied, what evidence was required, and how risk was assessed.

Engineers should be trained to slow down when needed. Speed without context increases risk. Correct handling means following runbooks where they exist, recognizing when a situation falls outside documented procedures, and escalating early rather than attempting to resolve high-risk issues independently.

Change Management

Change management is one of the highest-impact areas of Security Engineering work and one that is typically skipped during the onboarding process until a change is needed and the new hire has to figure out the process on their own. While the process takes time, the engineer must understand that change processes exist to control risk, not to create friction.

Training should start with an overview of the organization's change lifecycle, including scheduling, approval timelines, and review forums. From there, new hires should be walked through a complete change request example, including risk assessment, rollback planning, dependencies, and communication requirements. Attendance at change review meetings should also be included in the onboarding training. Observing how changes are challenged and approved helps build the judgement required to submit defensible changes later.

Engineers should understand:

- What changes must go through the change management process
- How to articulate security impact in plain language
- How to document outcomes and link related incidents or requests

Validation of Operational Readiness

Operational capability is validated through demonstrated performance, not completion of training sessions. Before expanding responsibility or access, the manager and mentor should confirm that the engineer can execute routine tasks reliably, follow established processes without prompting, and recognize when to pause and ask for help. Readiness is shown when an engineer can be trusted to operate within defined boundaries without constant oversight. Until that point, responsibility should remain scoped, reviewed, and intentionally constrained.

CREATE SOCIAL & ORGANIZATIONAL INTEGRATION

Technical capability alone is not sufficient for effective Security Engineering. Engineers operate inside a complex organizational system that includes decision authority, informal escalation paths, cross-functional dependencies, and trust relationships. Without deliberate integration into that system, new hires may technically perform tasks while still operating at increased risk due to misunderstanding ownership, influence, or expectations.

The goal of this section is to ensure that the new hire understands how work actually moves through the organization, who to involve at different stages, and how to communicate security decisions in a way that is effective and appropriate for the audience. This section focuses on building context, trust, and predictable interaction patterns so that security work does not stall, escalate incorrectly, or bypass the right stakeholders.

Structured Mentorship

A structured mentorship relationship is the primary mechanism for accelerating organizational understanding while reducing early-stage risk. The mentor provides context that is difficult to document, such as how decisions are challenged, where flexibility exists, and which processes are rigid by necessity.

The mentor should be assigned before the new hire's first day and should understand that their role is not limited to answering questions. They are responsible for guiding the new hire through real scenarios, explaining why decisions were made, and reinforcing expectations around judgement, escalation, and documentation.

Mentor engagement should be intentionally front-loaded, with more frequent touchpoints early in onboarding. As the new hire demonstrates confidence and judgement, the cadence can taper naturally. Mentorship sessions should follow a defined but loose structure to ensure coverage of critical topics defined by the manager while still allowing space for questions and discussion. The objective of mentorship is not independence at any cost, but safe autonomy aligned with defined capability boundaries.

Security Team Integration

Early integration into the larger Security team reduces hesitation, improves communication, and prevents isolation. New hires should be introduced formally during a team meeting and given clarity on how the team is structured, how work is divided, and how decisions are made. This gives the new hire exposure to the different security functions they will interact with, such as Security Operations, Incident Response, GRC, and leadership. Understanding where responsibilities begin and end helps prevent duplicated effort, missed handoffs, and incorrect assumptions during incidents.

Subject matter experts within the team should be identified and should provide overviews of their areas of responsibility. This reinforces that collaboration is expected particularly when dealing with an unfamiliar system.

While the primary objective of these introductions is operational clarity, allowing team members to share personal context and interests should be included in introductions to normalize communication and lowers hesitation when collaboration is needed later. Security work is stressful and we need to remember that it is executable by people, not roles alone, and familiarity reduces friction and builds a sense of team during incidents.

Cross-Functional Relationship Building

Security Engineering work depends heavily on collaboration with non-security teams. New hires must understand not only who these teams are, but how to work with them effectively. Key partner teams typically include infrastructure, networking, enterprise architecture, IAM operations, IT service management, and change management. Early introductions should focus on how security interacts with each group, common friction points, and preferred communication channels. The goal is to prevent the new engineer bypassing the right stakeholder or escalating too early due to uncertainty. Engineers who understand the organizational landscape can move work forward while maintaining trust and alignment.

On-Call Readiness

On-call participation represents a significant step in operational trust and should be treated as such. New hires should be introduced early to the concept of on-call responsibilities, even if they will not participate immediately. Formal sign-off for on-call readiness should occur only after the engineer demonstrates technical competence, sound judgement, and consistent escalation behavior. This protects both the individual and the organization.

Training should cover the scope of on-call duties, expected response times, communication standards, documentation requirements, and escalation expectations. Engineers should understand what ownership means during an incident, including when they are expected to act, coordinate, or hand off responsibility. Shadowing during on-call rotation is strongly recommended before formal participation if possible. This allows the new hire to observe real-world decision making under pressure and understand how incidents evolve beyond runbooks.

Communication Norms and Professional Judgement

Security Engineers are often required to communicate risk, deny requests, or challenge assumptions. New hires should be trained not only on what decisions to make, but how to communicate them effectively. This includes setting expectations for written communication in

tickets, chat platforms, and email, as well as verbal communication during meetings and incidents. Engineers should understand the difference between technical explanation and executive-level messaging, and when each is appropriate. Poor communication can undermine otherwise correct security decisions. Clear, professional, and context-aware communication reduces frictions, builds credibility, and improves long-term outcomes.

Validation of Organizational Integration

Successful integration is demonstrated when a new hire can navigate the organization without hesitation, engage the right stakeholders and the right time, and communicate decisions clearly and confidently. If gaps remain, they should be addressed deliberately through targeted exposure and guided discussion. Organizational uncertainty is a risk multiplier and should be treated with the same seriousness and technical gaps.

Managers and mentors should validate that the engineer understands:

- Who owns which systems and decisions
- When to escalate versus when to proceed independently
- How to collaborate effectively across teams
- How to represent security concerns in a way that is constructive and defensible

MEASURE PERFORMANCE

Measuring onboarding performance is not about scoring a new hire or creating artificial KPIs. The purpose of this section is to determine whether the onboarding framework is doing what it is supposed to do, which is to reduce ambiguity, enable safe contribution, and accelerate time-to-effectiveness without increasing risk. Measurement should focus on demonstrated capability, decision quality, and operational readiness, not activity volume or subjective impressions.

Performance measurement in this framework is intentionally lightweight and evidence-driven. It relies on milestone validation, observable operational signals, and structured feedback rather than continuous tracking or micromanagement. The outcome should be a clear answer to a simple question at each stage of onboarding – Can this engineer safely take on the next level of responsibility?

Milestone Validation

The primary mechanism for measuring onboarding performance is milestone validation at the 30, 60, and 90-day marks. These reviews should be structured, documented, and tied directly to the expectations defined earlier in this framework.

Milestone reviews are not generic check-ins. Each review should assess whether the new hire has met the specific outcomes defined for that stage, using concrete examples from their work. Validation should focus on demonstrated behavior and output, such as the ability to navigate systems independently, follow runbooks accurately, document decision clearly, and escalate appropriately when risk or ambiguity is present.

At each milestone, the manager and mentor should explicitly confirm:

- Which responsibilities the engineer can now perform independently
- Which activities still require review or supervision
- Whether capability boundaries should change (for example, moving tasks from Yellow to Green)
- Any gaps that present operational or security risk if left unaddressed

If milestones are not met, the goal is not to penalize the engineer, but to identify whether the issue is access, documentation quality, unclear expectations, or a mismatch between role definition and reality. Milestones should only be advanced when capability is demonstrated, not because a length of service has been reached.

Operational Metrics

Operational metrics provide supporting signals that complement milestone reviews. These metrics should be used carefully and interpreted in context. They are indicators, not targets. Useful operational signals may include:

- Ability to resolve low-risk tickets independently without rework
- Quality and completeness of ticket notes and change documentation
- Adherence to defined workflows and escalation paths
- Reduction in mentor intervention over time
- Consistency in applying runbooks and policies correctly

Metrics should never incentivize speed at the expense of judgement. For example, ticket volume alone is not a meaningful indicator of readiness if decisions are poorly documented or escalations are missed. When metrics are used, they should reinforce disciplined execution, not throughput.

Role Clarity and Confidence Check

A short role clarity check should be conducted as part of the 60 or 90-day review. This does not need to be a formal survey, but it should be structured enough to surface gaps early.

The new hire should be able to clearly articulate:

- What systems and workflows they are responsible for
- What decisions they can make independently versus what requires review
- Where their authority ends and escalation is required
- How their work supports broader security and risk objectives

If the engineer cannot explain these boundaries clearly, it is a signal that expectations, documentation, or mentoring need adjustment. Ambiguity at this stage is a risk, not a normal part of onboarding.

Remediation & Acceleration Path

Not all onboarding paths are linear. This framework explicitly support s both remediation and acceleration when appropriate. The goal in both cases is the same. The intent is to align responsibility with demonstrated capability while maintaining the organization's risk posture.

If gaps are identified, remediation should be targeted and time-bound. This may include additional runbook walkthroughs, focused documentation updates, shadowing specific workflows, or temporarily narrowing capability boundaries. Remediation plans should be documented and reviewed at the next milestone to confirm closure.

On the other hand, if an engineer demonstrates capability ahead of schedule, the framework should allow for accelerated responsibility in a controlled way. This might include earlier ownership of Green-zone tasks, expanded system access with review, or participation in more complex changes under supervision. Acceleration should still follow the same validation principles and never bypass defined approval or risk controls.

Using Measurement to Improve the Framework

Performance measurement is not only about the individual. It can also provide a feedback loop for the onboarding framework itself. Patterns across multiple hires, such as repeated delays in access, consistent confusion around a workflow, or frequent remediation plans in the same area, are signals that the framework needs adjustment.

Managers should periodically review onboarding outcomes across roles to identify systemic issues. Improvements should focus on clarifying expectations, strengthening documentation, and refining milestones rather than adding more process. Measurement is successful when it leads to fewer surprises, faster ramp-up, and safer delegation over time.

CONTINUOUS IMPROVEMENT PROCESS

This onboarding framework is designed to be a living system. Security Engineering work changes quickly. New tools are introduced, workflows evolve, and the threat landscape shifts in ways that affect how teams operate. A static onboarding document becomes outdated almost as soon as it is published. To remain effective, the framework must be continuously evaluated, refined, and realigned with the realities of the environment that it supports. This section is meant to create structure around these updates.

The continuous improvement approach recommended here builds from the same principles that strong Security Engineering programs are based on. During the development of this framework, several themes emerged from stakeholder interviews. These include inconsistent onboarding experiences, documentation drift, unclear role expectations, and friction in gaining access to required systems. These insights, combined with established improvement models used in areas like incident response, shaped a process that is intentionally lightweight but disciplined. The goal was not to overhaul the framework on a fixed schedule, but to maintain a steady rhythm of small, meaningful updates that keep it accurate and usable.

The Update Process

Maintaining the framework over time relies on a few core principles. Updates should be driven by evidence, such as feedback from new hires and mentors, outcomes from 30/60/90-day milestones, and signals from operational work. Improvement should be iterative, allowing the framework to evolve without creating too much churn. Changes need to remain transparent and documented so that teams understand what has been updated and why.

At a high level, the maintenance cycle follows a simple pattern. The framework is reviewed periodically to identify areas where expectations, documentation, or workflows no longer reflect current practice. Input from new hires, mentors, and cross-functional partners help highlight friction points or gaps. Once potential improvements are identified, they are prioritized based on impact and feasibility. Updates are then incorporated into the relevant sections of the framework. This may mean refining role descriptions, adjusting milestone expectations, or updating templates and checklists. After changes are published, they are communicated to mentors, managers, and any teams affected by the update. The impact of these changes is observed over time and used as input into the next review cycle.

Update Management

Contribution for this process can be shared but ownership should be clearly defined. The designated framework owner is responsible for coordinating reviews, maintaining version history, and ensuring updates are implemented consistently. Mentors and senior engineers may

contribute role-specific insights and validate technical accuracy. New hires can provide first-hand experience of issues that have arisen so they can be addressed quickly. Cross-functional partners can be included to ensure that updates remain aligned with organizational polices, provisioning workflows, and compliance requirements.

To keep the framework current without overwhelming the team, reviews should occur on a regular cadence. A quarterly review provides a structured opportunity to assess the framework as a whole. Reviewing after a new hire goes through the process is a good opportunity to capture timely insight from them. If there are significant changes to the tools, workflows, or regulatory requirements, these may trigger an ad-hoc update. All changes should be logged in the knowledge infrastructure to maintain a clear record of what was updated and why.

Ultimately, the continuous improvement process ensures that the onboarding framework remains relevant, accurate, and aligned with the evolving needs of the Security Engineering team. By treating onboarding as an adaptable system rather than a one-time deliverable, the organization strengthens its ability to bring new engineers up to speed efficiently and consistently, regardless of how the environment changes over time.

Revision Number	Reason for Revision	Revised By	Revision Date
1.0	Original Publication	Dustin Dishner	03/19/2026